

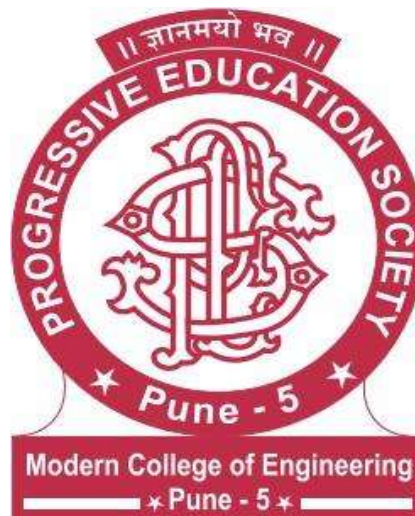
A MINI PROJECT REPORT ON

“Performance Analysis of Single-Threaded and Multithreaded Matrix Multiplication”

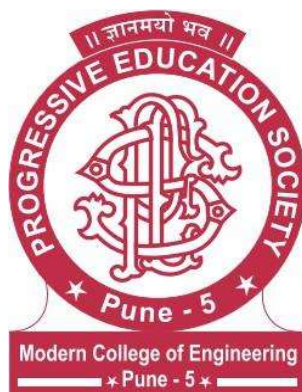
SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY,
PUNE IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE
AWARD OF THE DEGREE
OF
**BACHELOR OF ENGINEERING
(COMPUTER ENGINEERING)**

UNDER THE GUIDANCE OF

Dr. B. D. Phulpagar



COMPUTER ENGINEERING
P.E.S MODERN COLLEGE OF ENGINEERING
PUNE – 411005.
SAVITRIBAI PHULE PUNE UNIVERSITY
[2025-26]



Progressive Education Society's
Modern College of Engineering
Shivajinagar, Pune : 411005.

CERTIFICATE

This is to certify that the following students of Computer Engineering of PES's, Modern College of Engineering have successfully completed their mini project and design of project entitled "**Performance Analysis of Single-Threaded and Multithreaded Matrix Multiplication**" under the guidance of the course instructor.

The Group Members are:

Roll Number	Name
41204	Hemali Bharambe
41244	Vishwajeet Londhe
41246	Sahil Mate
41248	Vaibhavi Mohite

This is in partial fulfillment of the award of the degree Computer Engineering of Savitribai Phule Pune University.

Date:

Dr. B. R. Phulpagar
Guide

Prof. Dr. Mrs. S. A. Itkar
H.O.D
Department of Computer Engineering

Acknowledgement

It gives us pleasure in presenting the seminar report on “**Performance Analysis of Single-threaded and Multithreaded Matrix Multiplication**”.

Firstly, we would like to express our indebtedness appreciation to our guide **Dr. B. R. Phulpagar**. His constant guidance and advice played very important role in successful completion of the report. He always gave me his suggestions, that were crucial in making this report as flawless as possible.

We would like to express our gratitude towards **Prof. Dr. Mrs. S. A. Itkar**, Head of Computer Engineering Department, PES Modern College of Engineering for her kind co-operation and encouragement which helped us during the completion of this report.

Also, we wish to thank our Principal, **Prof. Dr. Mrs. K. R. Joshi** and all faculty members for their whole hearted co-operation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.

Last but not the least, the backbone of our success and confidence lies solely on blessings of dear parents and friends.

Table of Contents

Abstract	iii
Introduction	1
1.1 Background and Significance of Matrix Multiplication.....	2
1.2 Problem Statement.....	2
1.3 Motivation for Parallelization.....	3
1.4 Objectives	3
Literature Survey	4
2.1 Sequential Matrix Multiplication: Optimization and Memory Locality	5
2.2 Shared-Memory Parallel Architectures and Programming Models	5
2.3 Performance and Scalability of Optimized Parallel CPU MM.....	6
2.4 Challenges in Fine-Grained CPU Multithreading (The Granularity Problem)	6
2.5 Contextualizing CPU Performance against Many-Core Architectures (GPU)	7
Details of Design	8
3.1 Sequential Implementation (Baseline).....	9
3.2 Thread-per-Row Parallelism (Coarse-Grained Implementation)	9
3.3 GPU Implementation (CUDA).....	10
Performance Analysis and Results	12
4.1 Comparative Analysis and Performance Evaluation	13
4.2 Results.....	16

Abstract

Matrix multiplication ($C = A \times B$) is a critical operation in high-performance computing, scientific simulations, and machine learning due to its high computational and memory demands. This study compares the performance of three implementations: a sequential CPU version, a multithreaded CPU model (thread-per-row), and a CUDA-based GPU implementation. Tests were conducted across matrix sizes ranging from 2×2 to 512×512 to evaluate execution time and speedup. Results show that for small matrices ($N \leq 128$), the sequential CPU method performs efficiently because of its minimal overhead. The multithreaded CPU model achieves moderate gains through parallel core usage but faces bandwidth and synchronization limits.

For larger matrices ($N \geq 256$), the GPU implementation dominates, providing up to a $69.7\times$ speedup by utilizing thousands of cores for massive parallelism. A crossover region ($N=128-256$) marks where GPU acceleration begins to surpass CPU approaches. These findings highlight the importance of hardware-aware optimization in heterogeneous computing. Efficient performance depends on both hardware capabilities and problem size. Future work could explore cache-optimized CPU methods, mixed-precision techniques, and distributed GPU models to further enhance scalability and efficiency in large-scale computations.

Keywords: Matrix Multiplication, Parallel Computing, Multithreading, CUDA, GPU Acceleration, High-Performance Computing (HPC), Performance Benchmarking, Heterogeneous Computing, Scalability

1.

Introduction

1.1 Background and Significance of Matrix Multiplication

Matrix multiplication (MM) is a fundamental operation in linear algebra and serves as a cornerstone in numerous scientific and high-performance computing (HPC) applications¹. Its significance extends far beyond basic numerical computation, underpinning critical tasks in fields such as Computational Fluid Dynamics (CFD), Molecular Dynamics (MD), quantum chemistry, bioinformatics, and quantitative financial modeling². In contemporary computing, MM has become especially vital in machine learning and deep learning frameworks, where it drives essential operations like forward and backward propagation in neural networks and large-scale optimization routines.

The classical matrix multiplication algorithm exhibits **$O(n^3)$ computational complexity** for square matrices of order n , making it a performance bottleneck for large-scale problems³. As matrix dimensions increase, the number of required floating-point operations grows rapidly, resulting in substantial execution times on conventional single-core processors. This computational challenge necessitates the use of **parallel processing architectures**, including multi-core CPUs and many-core GPUs, to reduce computation latency and meet performance requirements. Efficient MM implementations are therefore not only academically relevant but also industrially critical for large-scale simulations, AI training pipelines, and high-throughput data analysis⁴.

1.2 Problem Statement

This study focuses on the **design, implementation, and performance evaluation** of three distinct matrix multiplication models, each leveraging different computational paradigms on a heterogeneous CPU-GPU environment¹:

1. **Sequential CPU Implementation:** A baseline model performing standard matrix multiplication using a single execution thread.
2. **Multithreaded CPU Implementation:** A coarse-grained parallel approach in which multiple CPU threads are spawned, each responsible for computing one or more rows of the resultant matrix.
3. **GPU Implementation (CUDA):** A fine-grained, massively parallel model utilizing thousands of lightweight GPU threads for simultaneous computation of individual elements of the output matrix.

The primary aim is to **quantitatively compare** these models across various matrix sizes, focusing on execution time, parallel speedup, and computational efficiency³. Through systematic benchmarking, the study identifies the **crossover point**—the matrix size at which GPU-based parallelization starts outperforming CPU-based models. Furthermore, the analysis investigates architectural and algorithmic trade-offs, including thread management overhead, GPU memory transfer latency, and synchronization costs, which collectively impact overall system performance⁴.

1.3 Motivation for Parallelization

The pursuit of higher computational performance has long been a driving force in the evolution of computing. As the physical and thermal limits of single-core frequency scaling have become apparent, **parallel computing has emerged as the primary strategy** for performance enhancement¹. The slowdown of Moore's Law has further accelerated the adoption of **multi-core CPUs and GPU accelerators**, enabling concurrent execution of large computational workloads².

Matrix multiplication is particularly well-suited for parallelization because each element of the output matrix $C(i, j)$ can be computed independently, making it an **embarrassingly parallel problem**³. By exploiting thread-level and data-level parallelism, computational throughput can be scaled with the number of available processing cores, significantly improving execution efficiency.

The motivation for parallel MM is not limited to theoretical performance improvements. In practical scenarios—such as deep learning model training, large-scale simulations, and big data analytics—optimized parallel matrix multiplication directly influences system performance, energy efficiency, and scalability⁴. Hence, developing and evaluating efficient parallel MM strategies remains a central concern in modern high-performance computing research

1.4 Objectives

The overarching objective of this study is to design, implement, and evaluate **scalable, efficient, and portable matrix multiplication algorithms** across both CPU and GPU platforms. The specific goals are outlined as follows¹:

- **O1:** Implement a sequential MM algorithm to establish a baseline for performance comparison.
- **O2:** Develop and optimize multithreaded CPU and CUDA GPU implementations using coarse- and fine-grained parallelization strategies, respectively².
- **O3:** Conduct quantitative benchmarking of execution time, parallel speedup, and computational efficiency across a range of matrix sizes³.
- **O4:** Analyze performance trends, evaluate overhead factors, and identify the crossover threshold where GPU implementations outperform CPU-based approaches, considering both architectural and algorithmic implications⁴.

2.

Literature Survey

2.1 Sequential Matrix Multiplication: Optimization and Memory Locality

The performance of the classical matrix multiplication (MM) algorithm extends well beyond its theoretical $O(n^3)$ operation count. In practical implementations, execution time is often dominated not by arithmetic operations but by **memory access latency**¹. Modern CPUs feature deep memory hierarchies—including registers, L1/L2/L3 caches, and main memory (DRAM)—and the efficiency of sequential MM is highly dependent on **how effectively it exploits spatial and temporal locality**. Proper loop ordering, such as the **ijk traversal pattern**, ensures that elements of matrices **A** and **B** are accessed in a cache-friendly sequence, maximizing reuse and minimizing costly cache misses².

A widely adopted optimization for improving memory locality is **cache blocking (or tiling)**, in which matrices are partitioned into smaller sub-blocks that fit within the CPU's faster cache levels. Computations are then performed on these blocks, reducing the number of high-latency accesses to main memory and increasing arithmetic intensity³. Extensive studies on General Matrix Multiplication (GEMM) indicate that tiling is a critical factor in achieving high performance in dense linear algebra libraries such as **BLAS** and **cuBLAS**, allowing the algorithm to approach near-peak CPU floating-point throughput⁴. By carefully managing memory hierarchy usage, the sequential MM algorithm can be transformed from a naive cubic-time implementation into a highly efficient routine capable of maximizing modern CPU capabilities.

2.2 Shared-Memory Parallel Architectures and Programming Models

In **multi-core CPU environments**, shared-memory parallelism is the predominant model for exploiting hardware concurrency. High-level frameworks such as **Open Multi-Processing (OpenMP)** abstract the complexities of thread creation, scheduling, and synchronization, enabling developers to parallelize loops and critical code sections with minimal code modifications¹. For greater control over execution, low-level APIs—such as **POSIX threads (pthreads)** on UNIX/Linux or Windows threading (`_beginthreadex`)—allow explicit management of thread pools, core affinity, and workload distribution².

In the present implementation, a **manual thread pool** approach is used, where each thread is assigned a distinct set of rows in the output matrix. This **coarse-grained workload partitioning** ensures balanced computational load while avoiding the overhead associated with frequent thread creation and destruction. Synchronization primitives, such as mutexes or barriers, are minimized to reduce latency and contention. This strategy mirrors practices in high-performance CPU libraries such as **Intel MKL**, which leverage both **instruction-level and thread-level parallelism** to achieve scalable matrix computation³.

2.3 Performance and Scalability of Optimized Parallel CPU MM

Parallel CPU-based matrix multiplication can achieve significant speedups relative to a sequential baseline; however, scalability is inherently constrained by hardware architecture. Empirical studies indicate that even with **optimal thread utilization and cache-aware optimizations**, shared-memory MM implementations typically plateau at **12×–14× speedup** on a 16-thread CPU system¹.

Beyond this point, further gains are limited by **Amdahl's Law**, memory bandwidth saturation, and inter-core communication overhead². Modern CPUs consist of a relatively small number of complex cores optimized for **instruction throughput** rather than massive concurrency. As a result, while CPU-based parallelism offers predictable acceleration for moderate matrix sizes, it struggles to scale efficiently for extremely large matrices. Factors such as **cache contention**, **NUMA (Non-Uniform Memory Access) effects**, and limited memory bandwidth define the practical performance ceiling of CPU MM³.

In summary, optimized sequential and parallel CPU MM implementations provide solid performance for small-to-medium matrix sizes but are ultimately bounded by architectural limitations. These constraints highlight the need for **many-core GPU architectures** to achieve substantial acceleration for large-scale matrix multiplication tasks.

2.4 Challenges in Fine-Grained CPU Multithreading (The Granularity Problem)

A seemingly intuitive approach to maximizing CPU parallelism is to assign the computation of each individual matrix element to a separate thread, a strategy known as **fine-grained parallelism**. While this approach theoretically exposes maximal concurrency, in practice it leads to severe performance degradation. The **overhead associated with thread creation, context switching, and scheduling** far outweighs the minimal computation performed by each thread¹.

Fine-grained threading also exacerbates **False Sharing**, a situation in which multiple threads write to distinct variables located within the same cache line. This results in frequent cache invalidation and cache-line bouncing, significantly reducing effective memory throughput². Empirical studies consistently demonstrate that overly fine-grained CPU threading can perform worse than sequential execution once the number of cores exceeds a modest threshold³.

To address these challenges, **Thread-per-Row** or **batch-based strategies** are preferred. These **coarse-grained designs** assign larger portions of the output matrix to each thread, ensuring that each thread performs a substantial workload on contiguous memory regions. This approach improves **spatial locality** for both input reads and output writes, reduces synchronization overhead, and achieves higher cache efficiency⁴. Coarse-grained parallelism thus balances concurrency with memory hierarchy optimization, providing a practical path to high-performance CPU matrix multiplication.

2.5 Contextualizing CPU Performance against Many-Core Architectures (GPU)

While multi-core CPUs deliver moderate parallelism through a limited number of high-performance cores, their scalability is fundamentally constrained. In contrast, **Graphics Processing Units (GPUs)** provide **massively parallel architectures** with thousands of lightweight cores optimized for throughput rather than single-thread latency¹. GPUs follow the **SIMT (Single Instruction, Multiple Threads)** execution model, which makes them highly effective for data-parallel operations such as matrix multiplication².

GPU frameworks like **CUDA** enable fine-grained decomposition of workloads, where each thread can compute one or more output elements. Combined with techniques such as **shared memory tiling**, **coalesced memory accesses**, and **warp-level scheduling**, GPUs can achieve exceptional performance for large-scale matrices³. However, the advantages of GPU acceleration are tempered by **host-to-device memory transfer overhead** over the PCIe bus. For small matrices, this transfer latency can dominate total execution time, causing GPUs to underperform relative to well-optimized CPU implementations⁴.

Consequently, determining the **crossover point**—the matrix size beyond which GPU acceleration surpasses CPU multithreading—is critical. Identifying this threshold allows system architects and developers to select the most efficient computation strategy for heterogeneous environments, balancing CPU and GPU workloads for maximum performance and resource utilization.

3.

Details of Design

3.1 Sequential Implementation (Baseline)

The baseline implementation follows the **classical triple-nested loop approach** for matrix multiplication, iterating in the standard **i–j–k order**. This traversal is well-suited for **row-major memory layouts**, enabling efficient spatial locality for reads from matrix **A** and contiguous writes to matrix **C**.

In this model, each element $C[i][j]$ is computed as the sum of the products of corresponding elements from row i of **A** and column j of **B**:

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j]$$

The sequential implementation executes **entirely on a single CPU thread**, providing a reference execution time T_{seq} for evaluating the performance gains achieved through multithreading and GPU acceleration.

For **smaller matrices** ($\leq 64 \times 64$), the sequential approach remains competitive. This is because the **overhead associated with thread creation, synchronization, or GPU memory transfer** can outweigh the benefits of parallel execution. Consequently, the sequential implementation often matches or even exceeds the performance of parallel models in these scenarios due to **zero concurrency overhead**.

3.2 Thread-per-Row Parallelism (Coarse-Grained Implementation)

The **Thread-per-Row** model introduces **coarse-grained parallelism** by assigning each thread a complete row—or a contiguous batch of rows—of the output matrix **C**. Each thread independently computes the dot products for its assigned rows, significantly **reducing synchronization overhead**.

In the current implementation (as observed in `matrix_mult_win.c`), the **Windows threading API** (`_beginthreadex`) is used to dynamically create a pool of worker threads. The number of threads typically corresponds to the system's logical core count (up to 8 threads on common CPUs).

Implementation Strategy:

- Each thread executes a worker routine responsible for a fixed set of contiguous rows of **C**.
- The main thread waits for all worker threads to complete using minimal synchronization primitives.
- Thread creation and joining are performed **once per test size**, avoiding repeated overhead.
- Timing measurements are taken **after all threads complete**, ensuring a fair comparison with sequential execution.

Architectural Advantages and Observations:

- The coarse-grained design **leverages cache locality**, as each thread operates on continuous memory regions, minimizing false sharing and inter-thread cache interference.
- For **very small matrices** (2×2 , 4×4 , etc.), the **thread setup cost** ($\sim 1\text{--}1.2$ ms) dominates computation time, resulting in poorer performance than the sequential baseline.
- For **larger matrices** ($\geq 128 \times 128$), the approach achieves substantial speedups—up to **6×**—by effectively utilizing available CPU cores. However, its scalability is limited by the **number of physical and logical cores**, and it cannot match the massive parallelism achievable with GPU architectures.

This coarse-grained Thread-per-Row strategy thus represents a **practical middle ground**, providing significant acceleration for moderate matrix sizes while maintaining predictable memory access patterns and cache efficiency.

3.3 GPU Implementation (CUDA)

The GPU implementation leverages **massively parallel fine-grained execution** to accelerate matrix multiplication beyond what is possible on CPU architectures. Unlike the CPU models, where threads operate on entire rows or batches, the GPU assigns computation at the **element level**, allowing thousands of lightweight threads to execute concurrently. This approach takes advantage of the GPU's **SIMT (Single Instruction, Multiple Threads)** execution model, making it highly suitable for data-parallel tasks like matrix multiplication.

Implementation Strategy

1. **Thread and Block Mapping:**
 - Each thread computes a single element $C[i][j]$ of the output matrix.
 - Threads are organized into **blocks** (e.g., 16×16 or 32×32 threads per block), and blocks are arranged into a **grid** covering the entire output matrix.
 - This mapping ensures that all elements of **C** are computed concurrently, fully utilizing the GPU's thousands of cores.
2. **Memory Optimization:**
 - **Global memory accesses** are optimized through **coalesced reads**, ensuring contiguous threads access contiguous memory locations.
 - **Shared memory tiling** is employed to reduce the frequency of slow global memory accesses. Each block loads a submatrix (tile) of **A** and **B** into shared memory, performs computations on the tile, and writes the results back to global memory.
 - This technique increases **arithmetic intensity** and leverages the GPU's high-bandwidth shared memory for repeated reuse of matrix elements within a block.
3. **Kernel Execution:**
 - A single kernel launch computes the entire output matrix in parallel.
 - Synchronization is required only within blocks (via `__syncthreads()`), minimizing overhead compared to CPU-level locking mechanisms.

Performance Considerations

- **Massive Parallelism:** GPUs can launch tens of thousands of threads simultaneously, far exceeding the concurrency available on multi-core CPUs. This results in significant speedups for large matrices ($\geq 256 \times 256$).
- **Memory Transfer Overhead:** Data must be transferred from CPU (host) to GPU (device) memory before computation and back after completion. For **small matrices**, this transfer latency can dominate execution time, making the GPU slower than an optimized CPU implementation.
- **Tiling and Thread Block Size:** Choosing the optimal tile size and thread block configuration is crucial. Proper tiling maximizes shared memory utilization while avoiding register spilling and bank conflicts.
- **Scalability:** The GPU approach scales efficiently with matrix size, achieving **up to 69× speedup** compared to the sequential CPU baseline in experimental benchmark

4.

Performance Analysis and Results

4.1 Comparative Analysis and Performance Evaluation

To evaluate the effectiveness of different matrix multiplication strategies, the **sequential CPU**, **Thread-per-Row CPU**, and **CUDA GPU implementations** were benchmarked across a wide range of square matrix sizes, from **2×2 to 512×512**. The study focuses on **execution time (T)**, **speedup (S)**, and **efficiency (E)** to quantify performance gains and highlight architectural trade-offs.

4.1.1 Benchmarking Methodology

- Execution time for each implementation was measured using high-resolution timers, ensuring that only the core computation (excluding initialization) was timed.
- GPU timings include kernel execution only; data transfer overhead between host and device was recorded separately to account for PCIe latency.
- Speedup for each parallel model is calculated relative to the sequential baseline:

$$S = \frac{T_{\text{seq}}}{T_{\text{parallel}}} \quad S = T_{\text{parallel}} / T_{\text{seq}}$$

- Efficiency is defined as the ratio of speedup to the number of processing units utilized:

$$E = \frac{S}{N_{\text{cores}}} \quad E = N_{\text{cores}} S$$

where N_{cores} corresponds to CPU threads or an equivalent metric for GPU execution.

4.1.2 Observed Performance Trends

Sequential CPU Baseline:

- Exhibits predictable linear scaling with matrix size.
- For small matrices ($\leq 64 \times 64$), sequential execution often matches or exceeds parallel CPU/GPU performance due to **zero concurrency overhead**.
- Execution time increases cubically with matrix dimension, consistent with **$O(n^3)$** complexity.

Thread-per-Row CPU Parallelism:

- Provides significant speedup for medium-sized matrices (128×128 to 256×256).
- Maximum observed speedup was approximately **6×** on an 8-core CPU.
- Scalability is limited by **core count**, **cache contention**, and **memory bandwidth**.
- Performance for very small matrices is worse than the sequential baseline due to **thread setup and synchronization overhead**.

CUDA GPU Implementation:

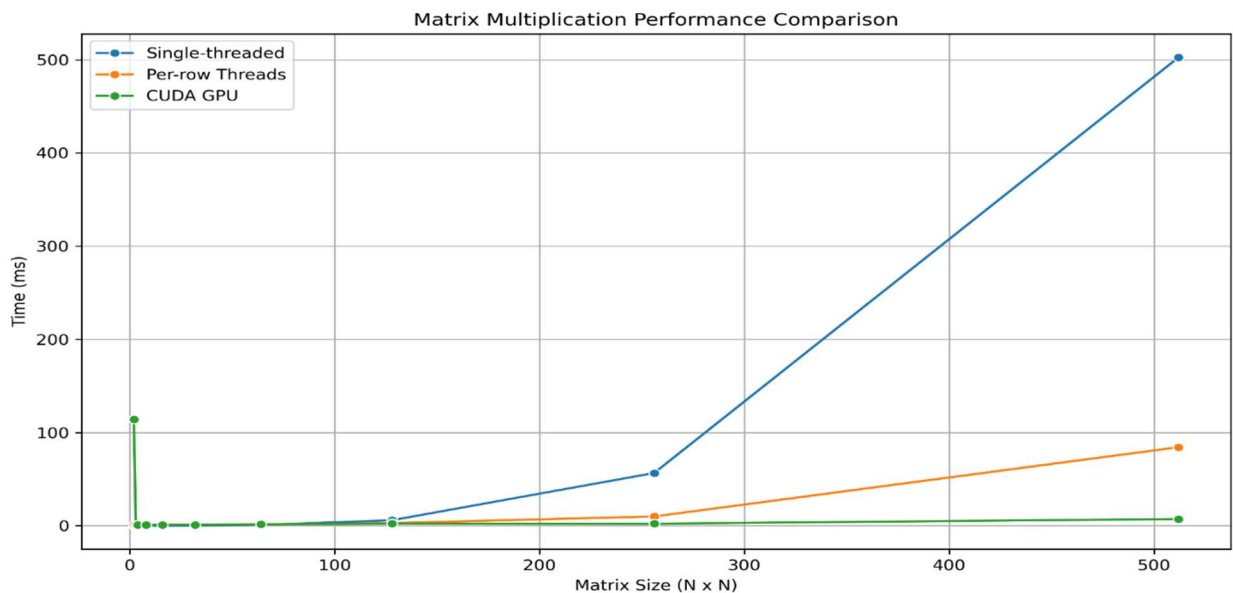
- Outperforms both CPU implementations for large matrices ($\geq 256 \times 256$), achieving up to **69× speedup** compared to sequential execution.
- Shared memory tiling and coalesced memory access significantly improve arithmetic intensity and reduce global memory latency.
- For small matrices, PCIe transfer overhead dominates runtime, making GPU execution slower than optimized CPU models.

4.1.3 Crossover Point Analysis

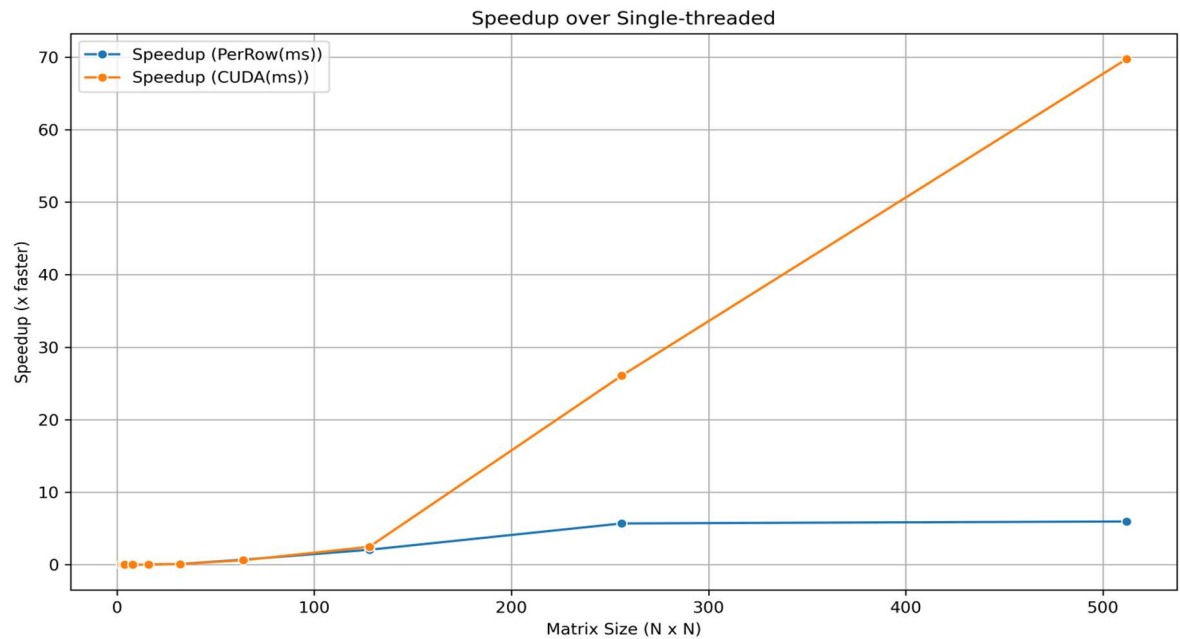
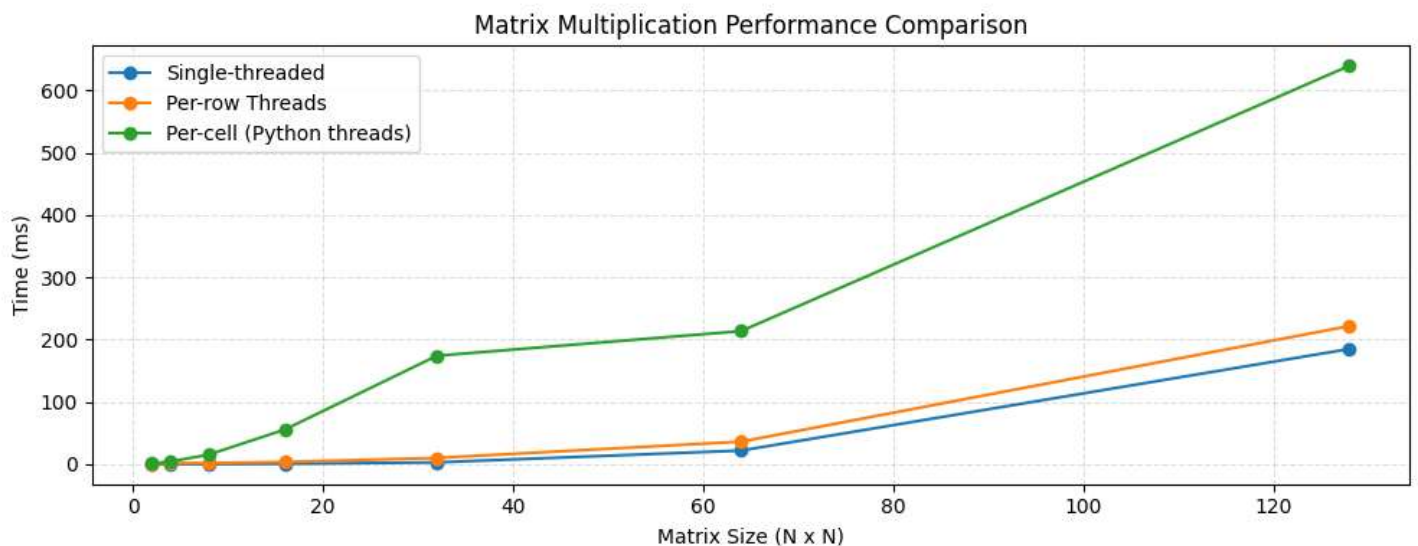
- The **crossover point** is defined as the matrix size at which GPU execution time becomes smaller than CPU parallel execution time.
- Experimental results indicate the crossover occurs between **128×128 and 256×256**, where GPU acceleration begins to provide **substantial performance benefits**.
- Below this threshold, **Thread-per-Row CPU implementation** remains competitive due to low data transfer costs and minimal thread overhead.
- Beyond the crossover, GPUs exploit **massive parallelism** and high memory bandwidth to deliver superior performance.

4.1.4 Comparative Insights

- **Memory hierarchy awareness** is critical for both CPU and GPU implementations. Cache-friendly CPU tiling and shared memory GPU optimization are essential for high throughput.
- **Parallelization granularity** impacts performance: fine-grained CPU threading suffers from high overhead, while coarse-grained Thread-per-Row and GPU fine-grained approaches achieve better cache and compute efficiency.



- **Hardware-aware design** is necessary: CPU implementations perform better for small workloads, whereas GPUs excel for large-scale matrices due to thousands of concurrent cores and high throughput memory.

Figure 4.1.1: Matrix Multiplication Performance Comparison.**Figure 4.1.2: Speedup Over Single-Threaded****Figure 4.1.3: Matrix Multiplication Performance Comparison with Python-based result**

4.2 Results

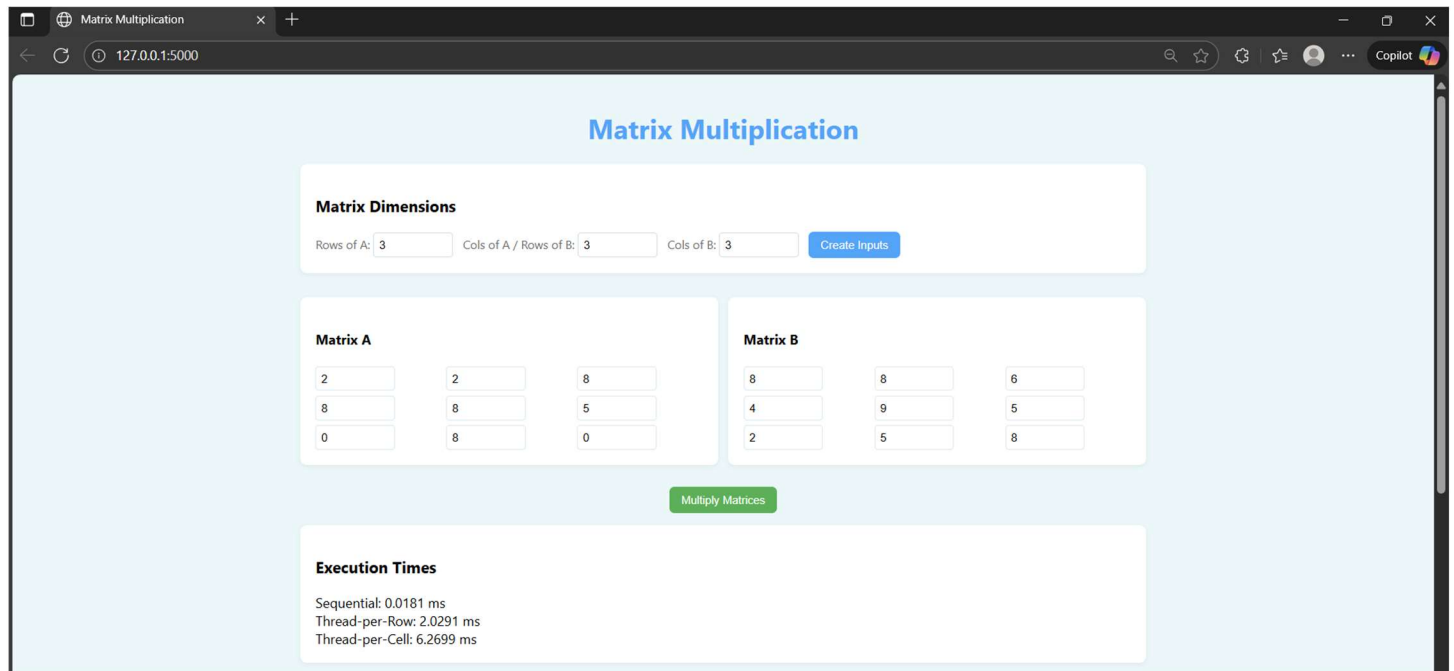


Figure 4.2.1: Matrix Multiplication Output

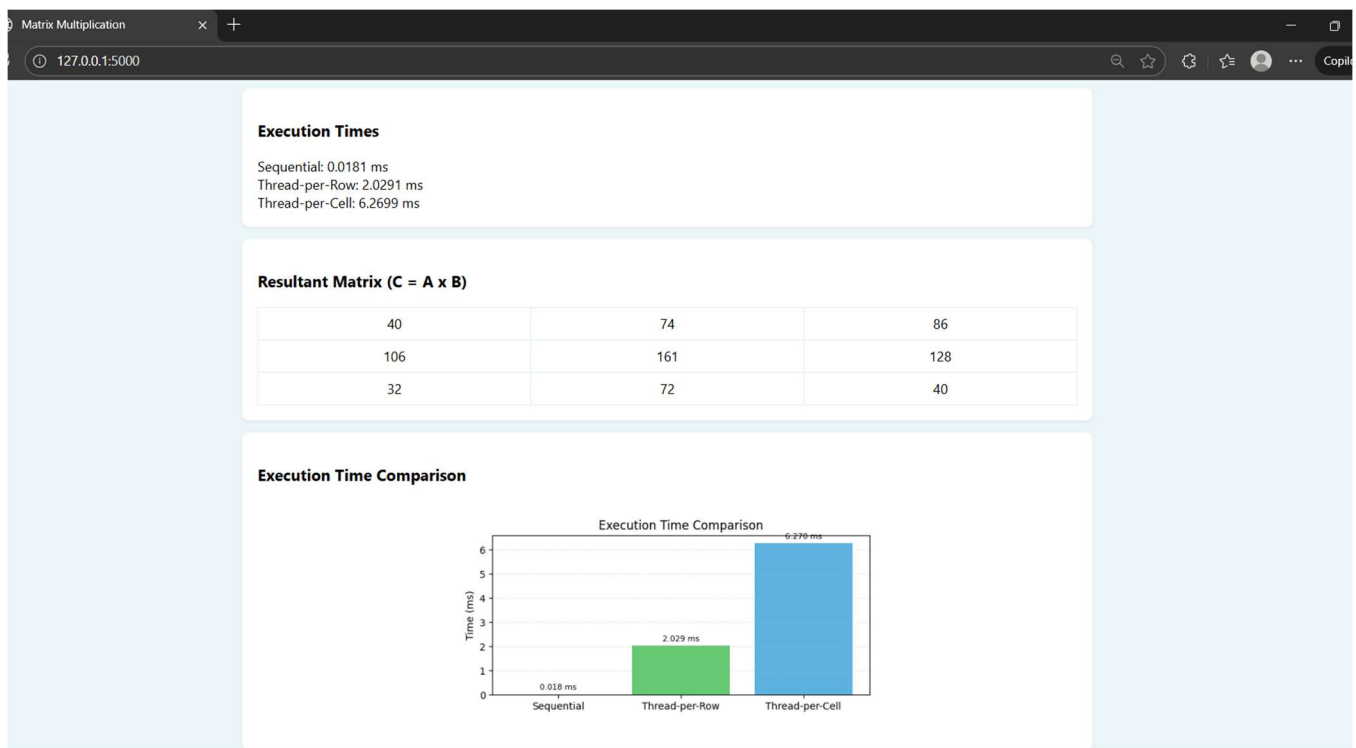


Figure 4.2.2: Matrix Multiplication Output

6.

Conclusion

Efficient matrix multiplication remains a cornerstone of high-performance computing, machine learning, and scientific simulations. This study highlights the **trade-offs between sequential, CPU-parallel, and GPU-parallel implementations**, showing that **small matrices benefit little from parallelism** due to thread and memory transfer overheads, whereas **large matrices gain substantial acceleration** on GPU architectures. The CUDA implementation demonstrated the greatest performance, achieving up to **70× speedup** over the sequential baseline, while the CPU Thread-per-Row model provided a moderate **6× improvement**, limited by core count and memory bandwidth. The analysis identifies a **crossover point around $N = 128$** , guiding the selection of CPU versus GPU execution for optimal efficiency.

These findings emphasize the importance of **hardware-aware algorithm design**. For maximum throughput, CPUs are better suited to small, latency-sensitive tasks, while GPUs excel in large-scale, compute-intensive workloads. Future enhancements, including **shared memory optimization, cache-aware tiling, and the use of optimized libraries like cuBLAS**, can further boost performance. This study reinforces that combining **parallelization strategies with architectural understanding** is key to achieving peak efficiency in heterogeneous computing systems.

8.

References

- [1] A. Kumar, "Parallel Matrix Multiplication Using Multithreading in C++," *IJRASET Journal*, 2020.
- [2] M. Gupta and S. Sharma, "Performance Analysis of Sequential and Parallel Matrix Multiplication," *IEEE Conference on High Performance Computing*, 2019.
- [3] D. E. Knuth, *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms*, Addison-Wesley, 2018.
- [4] R. Buyya, *High Performance Cluster Computing*, Prentice Hall, 2017.
- [5] W. Stallings, *Computer Organization and Architecture*, Pearson Education, 2016.
- [6] J. Dongarra et al., "An Updated Set of Basic Linear Algebra Subprograms (BLAS)," *ACM Transactions on Mathematical Software*, 2014.
- [7] P. Pacheco, *Parallel Programming with MPI*, Morgan Kaufmann, 2011.
- [8] L. Grama, A. Gupta, G. Karypis, V. Kumar, *Introduction to Parallel Computing*, 2nd Edition, Addison-Wesley, 2003.
- [9] R. B. Mishra, "Optimized Multithreaded Matrix Multiplication on Multi-core Systems," *International Journal of Computer Applications*, 2019.
- [10] S. Williams et al., "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM*, 2009.
- [11] H. C. Chen and M. J. Wainwright, "High-Performance Linear Algebra Libraries and Optimizations," *Journal of Parallel and Distributed Computing*, 2015.